

Creating a Barcode Generator in App Inventor 2

This tutorial will help you create a barcode generator app using App Inventor 2. It will be beneficial for readers joining now to refer to earlier issues to gain mastery over App Inventor 2.



1234500067890

Barcodes are quite commonplace in daily life. Because of things going digital nowadays, you see the check-out counter staff in shopping complexes and malls scanning the barcode pasted on the product rather than manually entering the data about a particular item. A barcode is an image that has bars of varying thickness and spaces, and retains information within that pattern. Different barcode images will have different information written into them. We have handheld barcode readers or smartphones that decode the information written in a barcode. A barcode can have information in any form like, names, numbers, addresses, the price, quantity, etc. The more the information in a barcode, the more dense will be the arrangement of bars in the image.

In this article, we will make a barcode generator application using App Inventor 2. By now, readers will have gained sufficient experience and hands-on practice with this tool. If this is the first time you are reading an article in this series, you can very well learn from this point itself, without any prior

programming or specialised knowledge. Apps developed with App Inventor can be uploaded to Google Play.

We can proudly say that we have mastered App Inventor in terms of the components that are available for use in the palette. And playing with the designer and block editor is a lot of fun.

Theme of the application

The theme is pretty simple and you probably have already got the idea from previous articles. We will make an Android application which will convert any written text to the barcode. The barcode generated can be saved to the SD or can be shared with other users via sharing components.

GUI requirement

For every application we have a graphical user interface or GUI, which helps the user to interact with the on-screen components. How each component responds to user actions is defined in the block editor section.

GUI requirements for Screen 1

1. **Label:** Labels are static text components that are used to display

some headings or markings on the screen.

2. **Button:** A button will let you trigger the event and is a very essential component.
 3. **Horizontal Arrangement:** This is a special component that keeps all the child components horizontally aligned within themselves.
 4. **Notifier:** This is used to display some instructions or give controls over your existing components. You will be able to explore its functionality in more detail as we implement it in our game.
 5. **Canvas:** This is the component for drawing and animation. You can place various animation objects over Canvas to control them via user actions. We will look at more details as we work with the application.
 6. **Text Box:** This is the component to take user inputs. Whatever text the user writes in the text box will be treated as input and can be used further. We will see more of it within the application itself.
- We will require the components listed in the table on the next page for this application. We will drag them on